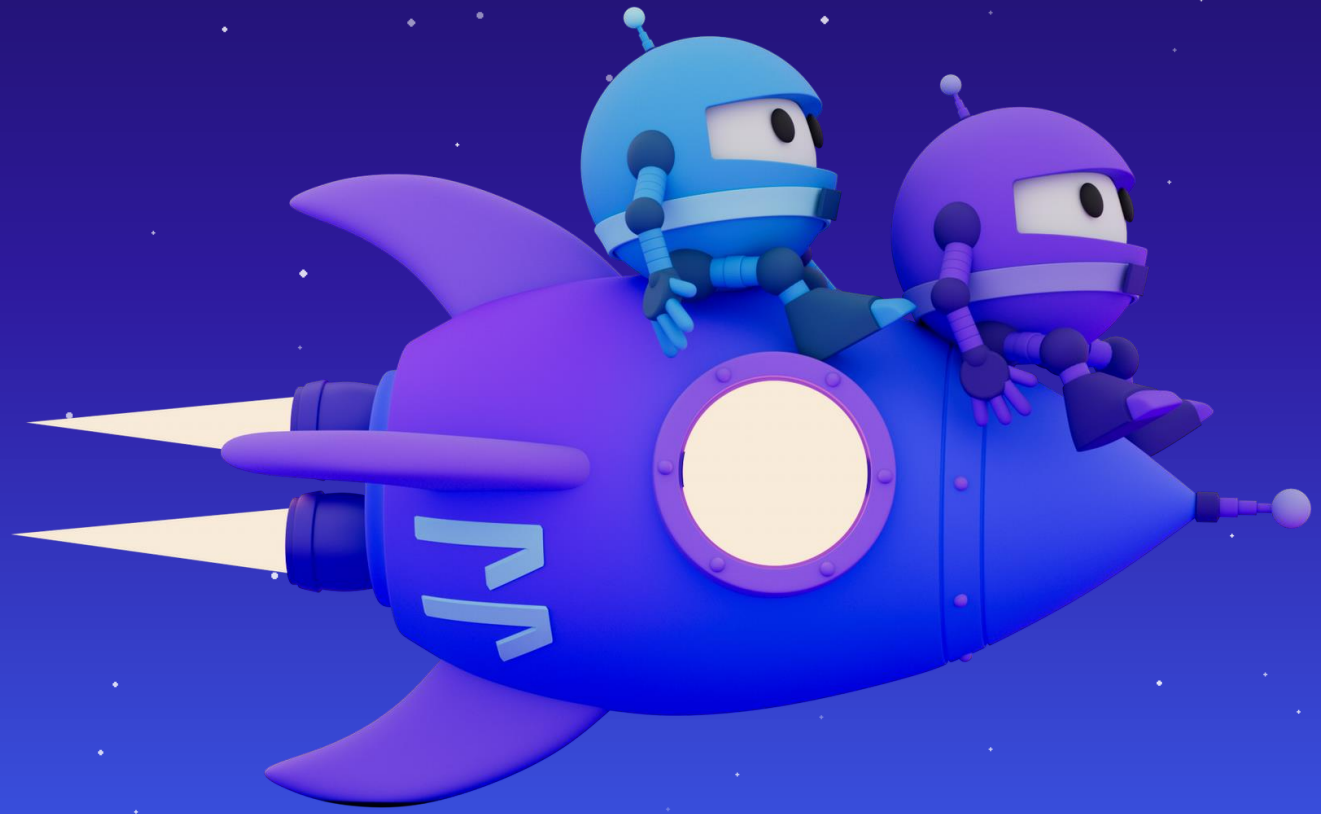
ASP.NET Core 11



OpenTelemetry



Martin Costello

Staff Software Engineer, Grafana Labs



Martin Costello
Staff Software Engineer @ Grafana Labs &
OpenTelemetry .NET SDK maintainer



@martincostello

@martincostello.com



Agenda

ASP.NET Core 11 💖 OpenTelemetry

- What is OpenTelemetry?
- What's new for observability in ASP.NET Core 11?
- What are Semantic Conventions?
- Demo
- Benefits of Semantic Conventions
- Contribute!

01 What is OpenTelemetry?



What is OpenTelemetry?

- An open standard for telemetry
- Vendor-neutral instrumentation for:
  - Logs
  - Metrics
  - Traces
  - Profiles
- Defines specifications for producing and consuming



02 What's new for observability in ASP.NET Core 11?



What's new for o11y in ASP.NET Core 11?

Subtitle

- Native OpenTelemetry tracing
- Metrics and tracing for Blazor components
- Metrics for MemoryCache
- Support for W3C Trace Context Level 2 sampling
- Configure trace sources with IConfiguration
- **Metrics and tracing implement HTTP Server Semantic Conventions**

03 What are Semantic Conventions?



”Semantic Conventions define a common set of attributes which provide meaning to data [...]”

What are Semantic Conventions?

- OTel Semantic Conventions (SemConv) defined in GitHub
  - open-telemetry/semantic-conventions
- Include logs, metrics, traces and resources
- Areas include:
  - HTTP (client and server)
  - Cloud Providers
  - Databases
  - RPC

Defined in YAML

Example

```yaml
- id: metric.http.server.request.duration
  type: metric
  metric_name: http.server.request.duration
  annotations:
    code_generation:
      metric_value_type: double
  brief: "Duration of HTTP server requests."
  instrument: histogram
  unit: "s"
  stability: stable
  extends: metric_attributes.http.server

- id: metric.http.client.request.duration
  type: metric
  metric_name: http.client.request.duration
  annotations:
    code_generation:
      metric_value_type: double
  brief: "Duration of HTTP client requests."
  instrument: histogram
  unit: "s"
  stability: stable
  extends: metric_attributes.http.client
```

04 Demo



Demo

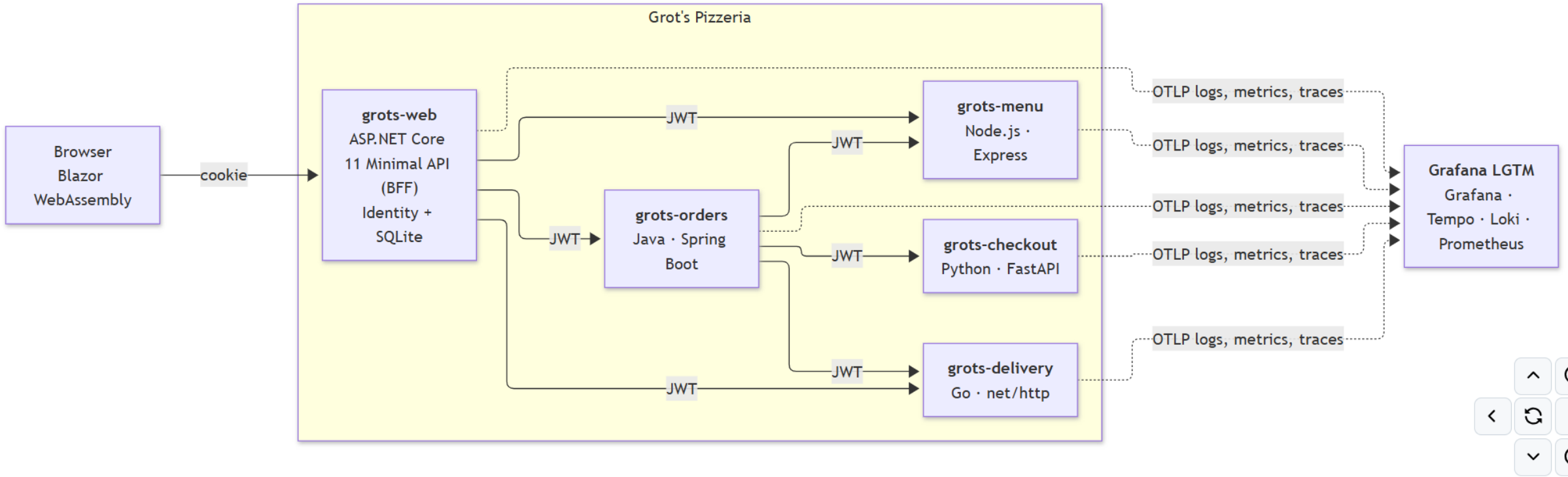
Grot's Pizzeria 🍕

Demo

Polyglot services + OpenTelemetry



Demo


Grot's Pizzeria
Browser
Blazor
WebAssembly
cookie
grots-web
ASP.NET Core
11 Minimal API
(BFF)
Identity +
SQLite
JWT
JWT
JWT
JWT
JWT
grots-orders
Java · Spring
Boot
grots-menu
Node.js ·
Express
grots-checkout
Python · FastAPI
grots-delivery
Go · net/http
OTLP logs, metrics, traces
OTLP logs, metrics, traces
OTLP logs, metrics, traces
OTLP logs, metrics, traces
OTLP logs, metrics, traces
Grafana LGTM
Grafana ·
Tempo · Loki ·
Prometheus



Configure OpenTelemetry

```
<!-- NuGet package references -->
<PackageReference Include="OpenTelemetry.Exporter.OpenTelemetryProtocol" Version="1.21.0" />
<PackageReference Include="OpenTelemetry.Extensions.Hosting" Version="1.21.0" />
```

Configure OpenTelemetry

```
{
  "Metrics": {
    "EnabledMetrics": {
      "Grots.Web": true,
      "Microsoft.AspNetCore.*": true,
      "System.Net.Http": true,
      "System.Net.NameResolution": true,
      "System.Runtime": true
    }
  },
  "Tracing": {
    "EnabledTracing": {
      "Microsoft.AspNetCore": true,
      "System.Net.Http": true
    }
  }
}
```

Configure OpenTelemetry

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddMetrics();
builder.Services.AddTracing(
    Tracing => tracing.AddConfiguration(
        builder.Configuration.GetSection("Tracing")));

builder.Services.AddOpenTelemetry()
    .ConfigureResource(resource => resource.AddService("grots-web"))
    .WithTracing(tracing => /* ... */)
    .WithMetrics(metrics => /* ... */)
    .WithLogging(logging => /* ... */)
    .UseOtlpExporter();

// Other application setup...

var app = builder.Build();

// Configure endpoints and middleware...

app.Run();
```

Demo
Components

- Sample code
  - martincostello/aspnetcore-11-loves-open-telemetry
- Dashboards:
  - LGTM: grafana/docker-otel-lgtm
  - Aspire: aspire.dev

Demo

- http://localhost:3000 - Telemetry dashboard
- http://localhost:8080 - Grot's Pizzeria Blazor web app

05 Benefits of Semantic Conventions



Benefits of Semantic Conventions

- Consistency of telemetry signals
  - Use the same dashboards and alerts whatever the ecosystem
- Open source
  - *"Lingua franca"* for observability
- Extensible for custom telemetry
  - Custom business telemetry and standardization
- Validation tools
  - Use tools like Weaver to enforce conventions

06 Contribute!



Contribute!

We 💖 contributions

- OTel .NET SDK: open-telemetry/opentelemetry-dotnet
- OTel SemConv: open-telemetry/semantic-conventions
- OTel Weaver: open-telemetry/weaver

Thank you!



Demo code



Feedback

ASP.NET Core 11 – OpenTelemetry